

Lab #2: System Call Implementation

Team Members

- **Manoj Manjunatha:** 862481448
 - **Vijay Ram Enaganti:** 862545736
 - **Subhash Bangalore Sateesha:** 862541575
-

Demo Video: <https://youtu.be/FH4qknTANck?si=C2DciZXeUDkRBfaS>

List of All Files Modified

Kernel

- `defs.h`
- `proc.h`
- `proc.c`
- `syscall.c`
- `syscall.h`
- `sysproc.c`

User

- `test.c`
- `user.h`
- `usys.pl`

Makefile

Changes Explained

`syscall.h`

Defines new syscall numbers for `SYS_sched_statistics` and `SYS_sched_tickets`, which are used to invoke their respective system calls.

```

kernel > C syscall.h > SYS_sched_tickets
13 #define SYS_sbrk 12
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 /*****/
24 #define SYS_sched_statistics 22 //assigning number 25 to new syscall sched_statistics
25 #define SYS_sched_tickets 23 //assigning number 25 to new syscall sched_tickets
26 /*****/
27

```

syscall.c

Manages system calls using an array of function pointers, where each index corresponds to a specific system call. Added the `sys_sched_statistics` pointer for the `sched_statistics` system call and the `sys_sched_tickets` pointer for the `sched_tickets` system call. Included the function prototypes for both `sched_statistics` and `sched_tickets`.

```

kernel > C syscall.c > ...
98 extern uint64 sys_write(void);
99 extern uint64 sys_mknod(void);
100 extern uint64 sys_unlink(void);
101 extern uint64 sys_link(void);
102 extern uint64 sys_mkdir(void);
103 extern uint64 sys_close(void);
104 extern uint64 sys_sched_tickets(void); // function prototype for sched_tickets syscall
105 extern uint64 sys_sched_statistics(void); // function prototype for sched_statistics syscall
106
107 // An array mapping syscall numbers from syscall.h
108 // to the function that handles the system call.
109 static uint64 (*syscalls[])(void) = {
110 [SYS_fork] sys_fork,
111 [SYS_exit] sys_exit,
112 [SYS_wait] sys_wait,
113 [SYS_pipe] sys_pipe,
114 [SYS_read] sys_read,
115 [SYS_kill] sys_kill,
116 [SYS_exec] sys_exec,
117 [SYS_fstat] sys_fstat,
118 [SYS_chdir] sys_chdir,
119 [SYS_dup] sys_dup,
120 [SYS_getpid] sys_getpid,
121 [SYS_sbrk] sys_sbrk,
122 [SYS_sleep] sys_sleep,
123 [SYS_uptime] sys_uptime,
124 [SYS_open] sys_open,
125 [SYS_write] sys_write,
126 [SYS_mknod] sys_mknod,
127 [SYS_unlink] sys_unlink,
128 [SYS_link] sys_link,
129 [SYS_mkdir] sys_mkdir,
130 [SYS_close] sys_close,
131 [SYS_sched_tickets] sys_sched_tickets, // sys_sched_tickets is a pointer to sched_tickets syscall
132 [SYS_sched_statistics] sys_sched_statistics, // sys_sched_statistics is a pointer to sched_statistics sysca
133 };
134
135

```

sysproc.c

Implemented the system call functions `sys_sched_statistics` and `sys_sched_tickets`, both returning an integer value of 0. In `sys_sched_tickets`, the argument passed is set as the number of tickets for the calling process using the inbuilt function `argint`.

The `sched_statistics` system call prints, for each process:

1. PID

2. Name in parentheses
3. Ticket value
4. Number of times it has been scheduled to run.

```
kernel > C sysproc.c > ...
72 sys_kill(void)
73 {
74     return kill(pid);
75 }
76
77 // return how many clock tick interrupts have occurred
78 // since start.
79 uint64
80 sys_uptime(void)
81 {
82     uint xticks;
83
84     acquire(&tickslock);
85     xticks = ticks;
86     release(&tickslock);
87     return xticks;
88 }
89
90 //syscall sched_tickets to set the caller process's ticket value to the given parameter
91 //return value of this function is int (0)
92 uint64
93 sys_sched_tickets(void)
94 {
95     int n;
96     argint(0,&n); //n is the inout parameter from user level, here it is number of tickets for that process
97     return set_tickets_to_proc(n);
98 }
99
100 //syscall sched_statistics to print pid, name, tickets and ticks of the process
101 //this function returns int 0
102 uint64
103 sys_sched_statistics(void)
104 {
105     print_sched_statistics();
106     return 0;
107 }
108
109
110
111
```

proc.h

Added necessary variables (**tickets**, **ticks**, **stride**, and **pass**) to the Process Control Block (PCB) to support the new scheduling mechanism.

```

kernel > C proc.h > proc > tickets
84 // Per-process state
85 struct proc {
86     struct spinlock lock;
87
88     // p->lock must be held when using these:
89     enum procstate state; // Process state
90     void *chan;           // If non-zero, sleeping on chan
91     int killed;           // If non-zero, have been killed
92     int xstate;           // Exit status to be returned to parent's wait
93     int pid;              // Process ID
94
95     // wait_lock must be held when using this:
96     struct proc *parent; // Parent process
97
98     // these are private to the process, so p->lock need not be held.
99     uint64 kstack;        // Virtual address of kernel stack
100    uint64 sz;             // Size of process memory (bytes)
101    pagetable_t pagetable; // User page table
102    struct trapframe *trapframe; // data page for trampoline.S
103    struct context context; // switch() here to run process
104    struct file *ofile[NOFILE]; // Open files
105    struct inode *cwd;       // Current directory
106    char name[16];          // Process name (debugging)
107    int tickets;             // total number of tickets per process
108    int ticks;              // counter to the number of times a process has been scheduled to run
109    int stride;             // interval between selectons
110    int pass;              // virtual time index of next section
111 };
112 };
113

```

proc.c

- Implemented both the Lottery and Stride scheduling algorithms. Declared global variables `DEFAULT_TICKETS`, `MAX_INT`, and `STRIDE_CONST`, with `DEFAULT_TICKETS` and `STRIDE_CONST` initialized to 10000.

```

20
21 #define DEFAULT_TICKETS 10000
22 #define MAX_INT 2147483647
23 #ifdef STRIDE
24 #define STRIDE_CONST 10000
25 #endif
26

```

- In the `allocproc()` function, initialized the number of tickets and set the `ticks` value to 0. Also initialized the `stride` and `pass` values for each process.

```

33
34 // To reset count for each process
35 p->ticks = 0; //initializing ticks to 0 for each process
36 p->tickets = DEFAULT_TICKETS; //initializing default tickets(10000) to each process
37 #ifdef STRIDE
38 p->stride = STRIDE_CONST / p->tickets; //initializing stride of the process
39 p->pass = p->stride; //initializing pass of the process
40 #endif
41
42

```

- Added functionality to print the following information for each process:
 - PID
 - Process name (enclosed in parentheses)
 - Ticket count

4. Number of times the process has been scheduled to run.

```

}

// Print the number of times the process was scheduled.
void print_sched_statistics(void) {
    struct proc * p;
    for (p = proc; p < &proc[NPROC]; p++) { //loop through all the processes
        acquire(&p->lock);
        if (p->state != UNUSED) {
            //printf function to print pid,name,tickets and ticks of the process
            printf("%d(%s): tickets: %d, ticks: %d\n",p->pid, p->name,p->tickets, p->ticks);
        }
        release(&p->lock);
    }
}

```

- Added functionality to set the value of the argument passed as the number of tickets for the calling process.

```

880 // Function to set the tickets to each process if called
881 int set_tickets_to_proc(int n) {
882     struct proc * p = myproc();
883     p->tickets = n;
884     #ifdef STRIDE
885     p->stride = STRIDE_CONST / p->tickets;
886     p->pass = p->stride;
887     #endif
888     return 0;
889 }
890

```

- Added functionality to return the sum of the ticket count of all processes when called.

```

472 // This function returns total_ticket count of all processes.
473 int TotalTickets(void) {
474     struct proc * process;
475     int total_tickets = 0;
476
477     for (process = proc; process < &proc[NPROC]; process++) { //loop through all the processes
478         acquire(&process->lock);
479         if (process->state == RUNNABLE) { //if the state of the process is runnable
480             total_tickets += process->tickets; // we increase the total_ticket count by the ticket of that process
481         }
482         release(&process->lock);
483     }
484     return total_tickets;
485 }
486
487

```

- Implemented the Lottery Scheduler, invoked when the "LOTTERY" scheduler is selected. Used preprocessor directives to enable switching between different schedulers.

```

487
488 //LOTTERY SCHEDULER IMPLIMENTATION
489
490 #ifdef LOTTERY //directive to run this block of code for LOTTERY scheduler
491
492 void
493 scheduler(void)
494 {
495     printf("Lottery Scheduler Invoked\n");
496     struct proc *p, *q;
497     struct cpu *c = mycpu();
498     int winner_decided; //variable to check if winner is decided or not
499     int lucky; //variable to store the winning lottery ticket number
500     int ticket_count; //current ticket count
501
502     c->proc = 0;
503     for(;;){
504         // Avoid deadlock by ensuring that devices can interrupt.
505         intr_on();
506         winner_decided = 0;
507         q = proc;
508         // Pick the winning lottery number
509         lucky = rand() % TotalTickets();
510         ticket_count = 0;
511
512         for(p = proc; p < &proc[NPROC]; p++) {
513             acquire(&p->lock);
514             if(p->state == RUNNABLE) {
515                 if (p->tickets == 0)
516                     p->tickets = DEFAULT_TICKETS;
517                 ticket_count += p->tickets;
518                 // The process gets scheduled only if its ticket is in the winning range.
519                 if (ticket_count >= lucky && !winner_decided) {
520                     q = p;
521                     winner_decided = 1;
522                 }
523             } else {
524                 p->tickets = 0;
525             }
526             release(&p->lock);
527         }
528         //Winning process gets the cpu
529         p = q;
530         acquire(&p->lock);
531         // Switch to chosen process. It is the process's job
532         // to release its lock and then reacquire it
533         // before jumping back to us.
534         p->state = RUNNING;
535         c->proc = p;
536         swtch(&c->context, &p->context);
537         p->ticks++;
538         // Process is done running for now.
539         // It should have changed its p->state before coming back.
540         c->proc = 0;
541         release(&p->lock);
542     }
543 }
544 #endif //STRIDE //use processor directive to run this block of code for STRIDE scheduler

```

- Implemented the Stride Scheduler, invoked when the "STRIDE" scheduler is selected. Used preprocessor directives to switch between schedulers based on the selected scheduling policy.

```

543 }
544 #elif STRIDE //pre processor directive to run this block of code for STRIDE scheduler
545 // Stride Scheduler Implementation
546 void
547 scheduler(void)
548 {
549     printf("Stride Scheduling\n");
550     struct proc *p;
551     struct proc *min_pass_p;
552     struct cpu *c = mycpu();
553     int min = MAX_INT;
554
555     c->proc = 0;
556     for(;;){
557         // Avoid deadlock by ensuring that devices can interrupt.
558         intr_on();
559         min_pass_p = proc;
560         for(p = proc; p < &proc[NPROC]; p++) {
561             acquire(&p->lock);
562             if(p->state == RUNNABLE) {
563                 if(p->pass < min) {
564                     min = p->pass;
565                     min_pass_p = p;
566                 }
567             }
568             release(&p->lock);
569         }
570
571         p = min_pass_p;
572         acquire(&p->lock);
573         // Switch to chosen process. It is the process's job
574         // to release its lock and then reacquire it
575         // before jumping back to us.
576         p->state = RUNNING;
577         c->proc = p;
578         switch(&c->context, &p->context);
579         p->ticks++;
580         // Process is done running for now.
581         // It should have changed its p->state before coming back.
582         p->pass += p->stride;
583         min = MAX_INT;
584         c->proc = 0;
585         release(&p->lock);
586     }
587 }
588 #else
589

```

defs.h

Declared the `print_sched_statistics()` and `set_tickets_to_proc()` functions, invoked in `sysproc.c`. Also declared a `rand()` function used in the Lottery Scheduler and called from `proc.c`.

```

190
191
192 int          set_tickets_to_proc(int);          // function declaration of set tickets to process
193 void         print_sched_statistics(void);      // function declaration of sched statistics
194 unsigned short rand(void);                    // function declaration of random integer returning function
195
196

```

usys.pl

Added new system calls `sched_statistics` and `sched_tickets` at the user level.

```

38 entry(uptime);
39 entry("sched_tickets");          #entering sched_tickets as a system call
40 entry("sched_statistics");       #entering sched_statistics as a system call

```

user.h

Declared `sched_statistics` and `sched_tickets` system calls at the user level according to their return types and arguments.

```
24 int uptime(void);
25 int sched_tickets(int);           //declaring system call sched_tickets
26 int sched_statistics(void);       //declaring system call sched_statistics
27
```

Makefile

```
73
74 LAB2 = RR
75 CFLAGS += -D$(LAB2)
76
```

Schedulers Explained

Lottery Scheduling

Lottery scheduling is a probabilistic CPU scheduling algorithm in which each process is assigned a number of "tickets." These tickets represent the process's chance of being selected for execution. The more tickets a process holds, the greater its probability of being chosen. This method balances fairness with simplicity by giving all runnable processes a non-zero chance of execution while allowing prioritization through ticket distribution.

In our implementation:

1. Iterate through all processes in the **RUNNABLE** state to compute the total number of tickets currently held.
2. Generate a random number to simulate a lottery draw. The drawn number is taken modulo the total ticket count to ensure it falls within the valid range.
3. Traverse the list of runnable processes again, cumulatively summing their ticket counts. The process whose cumulative total surpasses the randomly generated number is selected for execution.

To maintain correctness and avoid race conditions, locking mechanisms are used during the scheduling process. Ticket totals are recalculated during each scheduling cycle to reflect any changes, ensuring consistent and fair selection.

This approach allows for a flexible and fair allocation of CPU time, where processes with higher ticket counts receive proportionally more CPU attention, yet all processes retain a chance to be scheduled.

Stride Scheduling

Stride scheduling operates based on virtual time. Each process is assigned a "stride," inversely proportional to the number of tickets it holds. The scheduler always selects the process with the lowest **pass** value for execution. The **pass** value is determined by the stride of the process and is incremented after each time the process completes its quantum.

In our implementation:

1. The **scheduler()** function in **proc.c** was modified to support stride scheduling.
2. From the group of processes in the **RUNNABLE** state, the process with the lowest **pass** value is identified and selected for execution.
3. Once executed, the process's **pass** value is incremented by its stride, reflecting the amount of virtual time that has passed for that process.

To avoid race conditions, spinlocks are used to synchronize access to shared resources. When a context switch occurs, the acquired lock is released to exit the critical region. The scheduler then continues distributing resources by iterating through the processes, ensuring fair and efficient execution based on their respective strides.

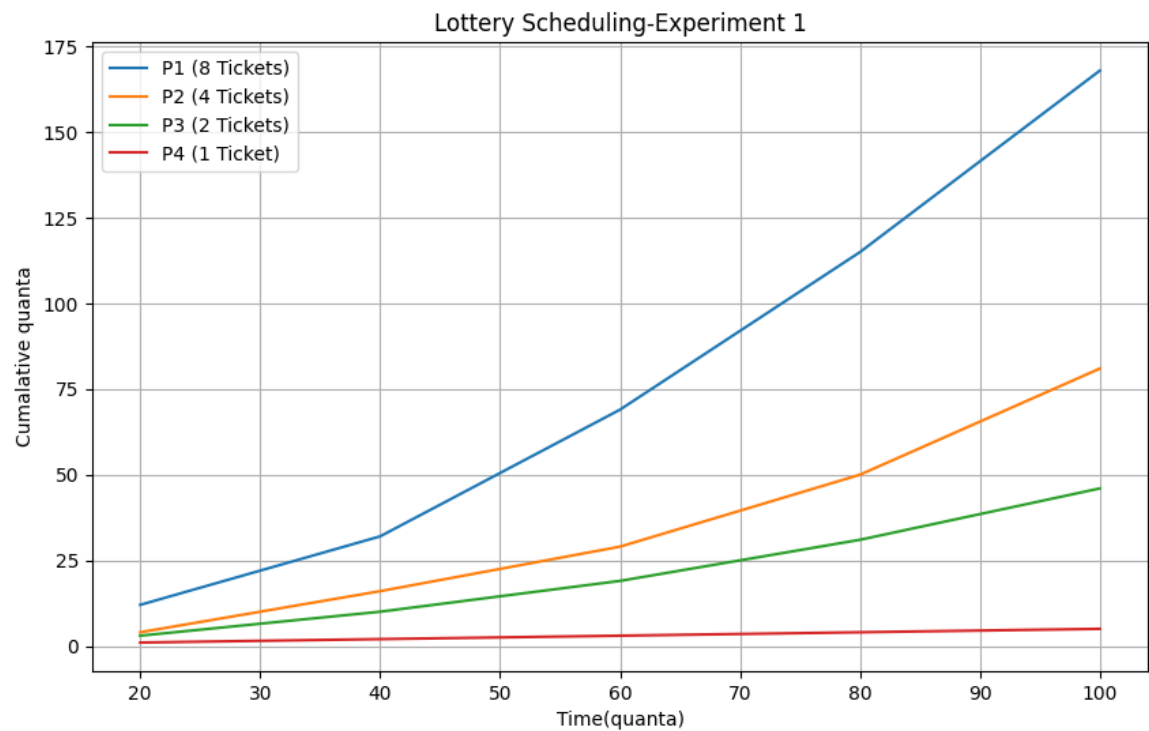
Experiments

We have run each experiment for 5 iterations and taken the average values to plot the graph.

Experiment 1: Four Processes with 8, 4, 2, and 1 ticket respectively.

Lottery Scheduling

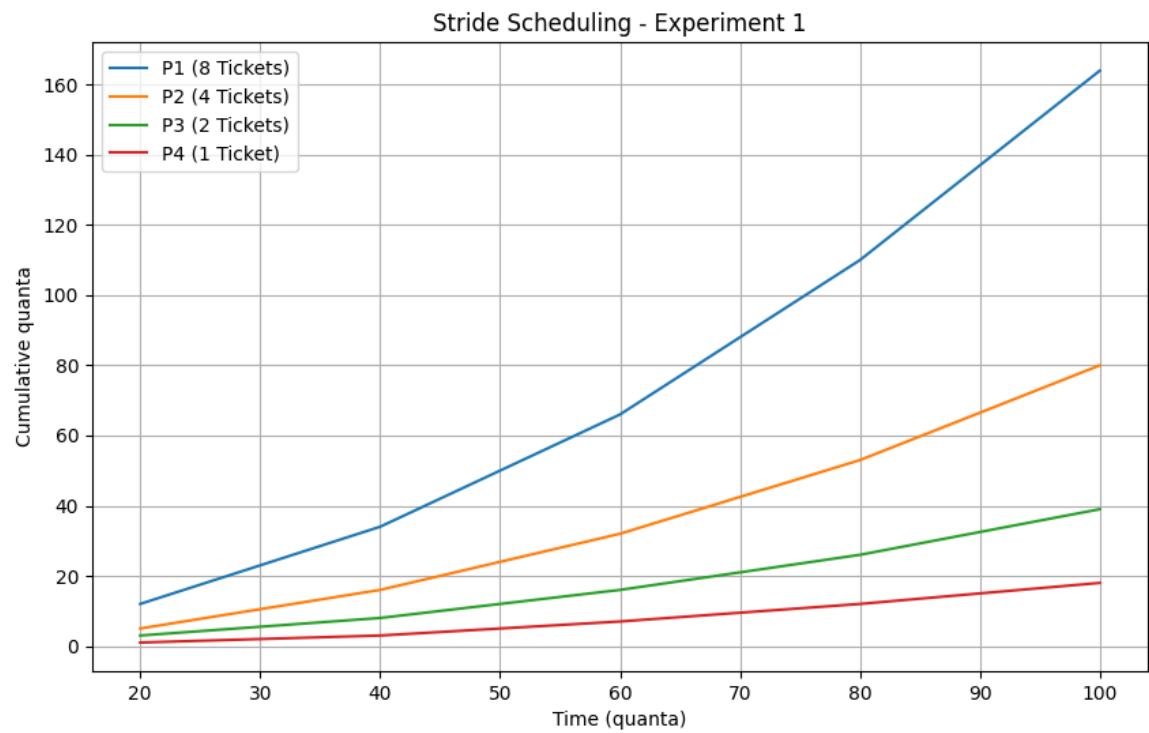
Time	P1 (8 Tickets)	P2 (4 Tickets)	P3 (2 Tickets)	P4 (1 Ticket)
20	12	4	3	1
40	20	12	7	1
60	37	13	9	1
80	46	21	12	1
100	53	31	15	1



Stride Scheduling

Time	P1 (8 Tickets)	P2 (4 Tickets)	P3 (2 Tickets)	P4 (1 Ticket)
------	----------------	----------------	----------------	---------------

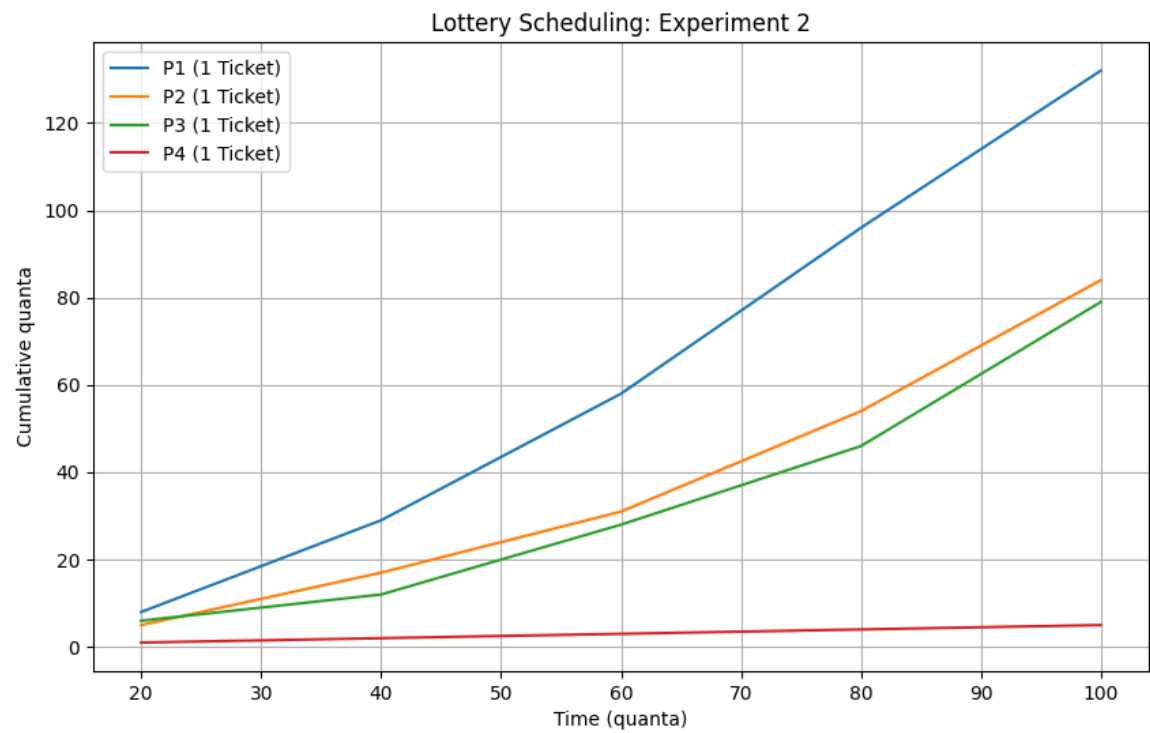
Time	P1 (8 Tickets)	P2 (4 Tickets)	P3 (2 Tickets)	P4 (1 Ticket)
20	12	5	3	1
40	22	11	5	2
60	32	16	8	4
80	44	21	10	5
100	54	27	13	6



Experiment 2: Four Processes with 1 ticket each.

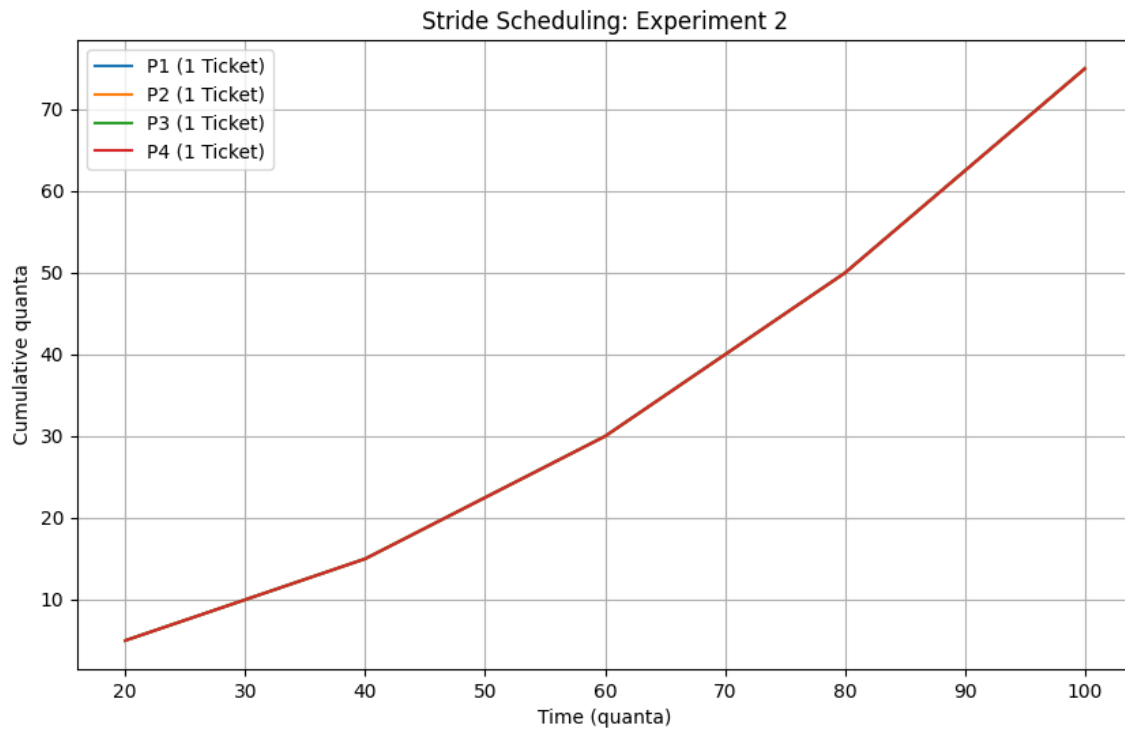
Lottery Scheduling

Time	P1 (1 Ticket)	P2 (1 Ticket)	P3 (1 Ticket)	P4 (1 Ticket)
20	8	5	6	1
40	21	12	6	1
60	29	14	16	1
80	38	23	18	1
100	36	30	33	1



Stride Scheduling

Time	P1 (1 Ticket)	P2 (1 Ticket)	P3 (1 Ticket)	P4 (1 Ticket)
20	5	5	5	5
40	10	10	10	10
60	15	15	15	15
80	20	20	20	20
100	25	25	25	25



- We observed that Lottery Scheduling offers a probabilistic guarantee, whereas Stride Scheduling provides a stronger, deterministic guarantee. Based on the results, Stride Scheduling tends to perform better than Lottery Scheduling when all processes are assigned an equal number of tickets. This is because, under equal ticket distribution, the Lottery Scheduler selects a process based on a random draw, which can lead to variability in scheduling. In contrast, Stride Scheduling selects the process using a deterministic calculation based on the pass value, resulting in more predictable and fair process selection. When tickets are distributed unevenly—for example, if one process is assigned only a single ticket—the behavior differs significantly between the two schedulers:
 - Lottery Scheduler: Processes with fewer tickets receive fewer CPU ticks and may suffer from resource starvation, especially if other processes hold a significantly higher number of tickets.
 - Stride Scheduler: Such processes still receive a fairer share of CPU time compared to Lottery Scheduling. Over time, even low-ticket processes are guaranteed CPU access, reducing the likelihood of starvation.
 - Overall, Stride Scheduling demonstrates more consistent and equitable behavior, particularly in scenarios with unequal ticket distribution.

Contributions

- **Manoj:** Implemented logic for the Lottery Scheduler and performed experiments for Lottery Scheduling.
- **Vijay:** Implemented logic for the Stride Scheduler and performed experiments for Stride Scheduling.
- **Subhash:** Implemented the logic for the system calls `sched_statistics` and `sched_tickets` as part of Lab 2, Part 1. Developed a function to return the total number of tickets across all processes. Additionally, contributed to report creation.

